

DATA MINING PROJECT REPORT

E-Commerce Store Management System

Database Design, Exploratory Data Analysis, Data Mining & Business Intelligence

Team Leader	Mohamed Khaled Mahmoud Ibrahim
Lab / Section	Lab 2
Report Phase	Phase 1 of 4: Database Design & Foundations
Submission Date	March 2026
Language	Formal Academic English
Classification	Academic Project Report — Confidential

*Prepared in fulfillment of the Data Mining Course Requirements
Military Technical College | Digital Pioneers Initiative | March 2026*

Executive Summary

This academic report presents a comprehensive, end-to-end data mining and analytics pipeline designed and implemented for a simulated E-Commerce platform. The project was conceived and executed by a team led by Mohamed Khaled Mahmoud Ibrahim under the academic auspices of the Military Technical College (MTC) and the Digital Pioneers Initiative (DEPI). The overarching objective was to architect, populate, and analytically interrogate a multi-table relational database while simultaneously deriving actionable business intelligence from a large-scale transactional dataset encompassing over 22,000 customer records spanning ten major Turkish metropolitan cities.

Project Scope and Objectives

The project was structured across four interrelated analytical phases, each building upon the outputs of its predecessor. In the first phase — detailed extensively in this report — the team designed a fully normalized, enterprise-grade relational database management system (RDBMS) using Microsoft SQL Server. This database, composed of fourteen distinct entity tables governed by a rigorously engineered Entity-Relationship Diagram (ERD), was designed to faithfully replicate real-world e-commerce operations including customer lifecycle management, order processing, inventory tracking, payment reconciliation, shipment logistics, and post-sale customer support.

The second phase constituted a deep Exploratory Data Analysis (EDA) using the Python ecosystem — specifically the Pandas, NumPy, Matplotlib, and Seaborn libraries — within a Jupyter Notebook environment. Over twenty analytically distinct questions were formulated and answered, probing the dataset across dimensions including revenue distribution, customer demographic profiles, product category performance, temporal sales trends, and the statistical

relationships between key numerical features. This phase surfaced several high-value insights that directly informed the business recommendations presented throughout this report.

The third phase leveraged the Orange Data Mining visual programming framework to construct a 35-widget end-to-end machine learning pipeline. This pipeline traversed the full spectrum of the Knowledge Discovery in Databases (KDD) process — from raw data ingestion and visual EDA through unsupervised clustering via K-Means algorithms to supervised classification models including Random Forest, Support Vector Machines (SVM), and Decision Trees. The predictive target was the identification of Returning Customers, a commercially critical classification task with direct implications for customer retention investment.

The fourth and final phase translated the structured analytical findings into interactive, executive-ready Business Intelligence dashboards constructed within Microsoft Power BI. Three distinct dashboards were developed: an Executive Operations Overview tracking aggregate revenue and geographic distribution; a Customer and Product Analytics dashboard dissecting category-level profitability; and a Satisfaction and Retention dashboard monitoring customer loyalty metrics, return rates, and delivery performance.

Key Findings and Business Implications

The analysis revealed several strategically significant findings. First, the Electronics product category consistently emerged as the dominant revenue driver across all analytical tools and methodologies. Whether assessed through SQL aggregate queries, Python correlation analysis, or Power BI visual dashboards, Electronics generated disproportionately high Total Revenue relative to transaction volume, yielding an average order value of approximately \$4,748 — nearly four times the overall dataset mean of \$1,211. This finding has direct implications for inventory investment prioritization and digital advertising budget allocation.

Second, the Pearson correlation analysis conducted during the EDA phase established a statistically robust positive correlation coefficient of 0.85 between Unit Price and Total Amount. This relationship, validated independently through Orange scatter plot visualizations, confirms that the primary lever for revenue maximization is premium product pricing strategy rather than volume-driven sales tactics. Quantity exhibited a substantially weaker correlation of 0.27, reinforcing this conclusion.

Third, the temporal analysis of revenue streams revealed pronounced seasonal variation, with a notable revenue peak concentrated in Q4 (October through December) consistent with global e-

commerce holiday spending behavior. A discernible trough was also identified in Q2, particularly in the months of April and May. These findings enable the operations and marketing teams to proactively align inventory restocking cycles, staffing levels, and promotional campaigns with anticipated demand fluctuations.

Fourth, the machine learning classification models trained in Orange demonstrated that customer return behavior can be predicted with meaningful accuracy, with the Random Forest model achieving the highest Area Under the Curve (AUC) score of 0.802 and an overall accuracy of 80.2%. The confusion matrix analysis confirmed that the model's false-positive and false-negative rates are within commercially acceptable thresholds for loyalty campaign targeting. Feature importance analysis identified Total Amount spent, Product Category preference, and City of residence as the three strongest predictors of customer return probability.

Fifth, the geographical distribution of orders — while predominantly concentrated in Istanbul, which accounted for approximately 1,672 total orders representing the largest single-city share — revealed meaningful demand nodes across all ten cities in the dataset. Ankara and Izmir constituted the second and third largest order volumes at 927 and 828 respectively, suggesting opportunities for targeted regional distribution center optimization.

Report Structure

This Phase 1 report is organized into the following primary components. Following this Executive Summary, Chapter 1 provides an in-depth exposition of the database architecture, covering the Entity-Relationship Diagram design rationale, the business rules and normalization principles applied, the complete Data Definition Language (DDL) implementation, and the data ingestion strategy. Subsequent phases of the report — to be submitted incrementally — will address the Python EDA phase, the Orange Data Mining machine learning pipeline, and the Power BI Business Intelligence dashboards respectively.

All technical implementations described herein were executed by the project team within the academic curriculum framework established jointly by the Military Technical College and the Digital Pioneers Initiative. The methodologies applied are consistent with industry-standard data engineering and analytics practices and are intended to demonstrate readiness for professional deployment in real-world data-intensive organizational contexts.

Report Phase: Phase 1 — Database Design & Foundations
Dataset Size: 22,049 transactional records across 10 Turkish cities
Database Tables: 14 normalized relational tables (SQL Server 2019)
Total Revenue Tracked: \$7.92M (Power BI Executive Dashboard)
Top Category: Electronics (\$3.7M total revenue)
ML Best Model: Random Forest (AUC = 0.802, Accuracy = 80.2%)
Tools Used: SQL Server, Python (Pandas/Seaborn), Orange, Power BI

Chapter 1: Database Analysis and Design (SQL Implementation)

This chapter presents a comprehensive technical exposition of the relational database management system designed and implemented as the foundational data infrastructure for the E-Commerce Store Management System. The database was engineered in Microsoft SQL Server 2019 and represents the first and most critical phase of the overall data pipeline. A robust, well-normalized database not only ensures the integrity and consistency of operational data but also provides the stable analytical foundation upon which all subsequent exploratory analysis, machine learning modeling, and business intelligence reporting are built.

1.1 Database Architecture and Core Design Philosophy

The database architecture adopted for this project adheres to the Third Normal Form (3NF) of relational database normalization, a fundamental principle in database engineering that eliminates transitive dependencies and ensures that every non-key attribute is functionally dependent solely upon the primary key of its respective table. This approach minimizes data redundancy, prevents update anomalies, and maintains referential integrity across the entire schema — properties that are essential in a high-throughput e-commerce transactional environment where simultaneous read and write operations are frequent and where data consistency directly impacts business operations such as inventory management, payment processing, and customer communications.

Beyond normalization, the architectural design was guided by several enterprise-grade principles. The schema was constructed to be horizontally scalable, meaning that as the e-commerce platform grows — adding new product lines, expanding geographically, or onboarding additional couriers and payment gateways — new tables and foreign key relationships can be appended without necessitating structural modifications to existing entities. This future-proof design philosophy is reflected in the deliberate use of `IDENTITY(1,1)` primary key columns, which ensure globally unique, auto-incrementing record identifiers that remain stable even under high-volume concurrent insertion workloads.

1.1.1 Business Rules and Operational Assumptions

The translation of real-world e-commerce business processes into a formal relational schema required the explicit articulation and encoding of several foundational business rules. These rules govern the permissible relationships between entities, define the constraints applied to data fields,

and establish the behavioral logic embedded within the database structure itself. The following business rules were formally adopted:

- **Customer Address Multiplicity:** A single customer record may be associated with multiple shipping and billing addresses, reflecting the real-world scenario in which customers maintain distinct home, work, and alternate delivery locations. This relationship is implemented as a one-to-many (1:M) association between the Customers table and the CustomerAddresses table, with the CustomerID foreign key in the CustomerAddresses table enforcing referential integrity.
- **Soft Deletion of Orders:** To preserve the completeness and auditability of financial records, no order record is ever permanently removed from the database through a hard DELETE operation. Instead, orders that are cancelled, returned, or otherwise voided are flagged through a status field transition to a 'Cancelled' state. This soft-deletion strategy is critical for regulatory compliance, financial auditing, and retrospective analysis of cancellation patterns.
- **Payment-Order Singularity:** Each order is associated with at most one definitive payment record, enforced through a strict one-to-one (1:1) relationship between the Orders table and the Payments table via a UNIQUE constraint on the OrderID foreign key in the Payments table. This constraint prevents duplicate payment records for a single order, which would introduce financial discrepancies.
- **Product-Category Exclusivity:** Each product is assigned to precisely one primary category, enforced through a non-nullable CategoryID foreign key in the Products table referencing the Categories table. This single-category constraint simplifies inventory tracking and ensures unambiguous category-level revenue aggregation in analytical queries.
- **Cart-to-Order Lifecycle:** The shopping cart entity persists independently of the order entity, allowing customers to maintain active carts across sessions prior to checkout. Upon order placement, the CartID is recorded in the Orders table as a foreign key, preserving the audit trail of the purchasing journey from cart creation through order completion.
- **Returns and Refunds Cascade:** Returns are linked to their originating orders and customers through foreign key relationships, and each return may optionally trigger a corresponding refund record. The Refunds table maintains a foreign key referencing the Returns table, establishing a traceable financial chain from product return through monetary reimbursement.
- **Support Ticket SLA Enforcement:** Customer support tickets carry priority classifications (Critical, High, Medium, Low) that correspond to Service Level Agreement (SLA)

response time thresholds enforced through application-layer logic. The database schema captures ticket creation timestamps, closure timestamps, and SLA breach flags to enable operational performance monitoring.

1.1.2 Normalization Analysis

The schema satisfies First Normal Form (1NF) through the elimination of repeating groups and the enforcement of atomic column values — for example, customer phone numbers, product descriptions, and city names each occupy discrete, single-valued fields. Second Normal Form (2NF) compliance is achieved by ensuring that all non-key attributes in composite-key junction tables (such as OrderItems and CartItems) are fully functionally dependent on the entire composite key rather than on a subset thereof. Third Normal Form (3NF) is satisfied by removing transitive dependencies — for instance, the CategoryName is stored exclusively in the Categories table and referenced by foreign key from the Products table, rather than being redundantly stored in the Products table itself.

This normalization discipline has a direct and measurable impact on database performance and analytical integrity. In the context of this project's dataset of 22,049 transactional records distributed across fourteen tables, the normalized schema reduces storage footprint by eliminating redundant string repetition (e.g., city names, category labels, payment method strings), enables efficient indexed lookups on primary and foreign key columns, and ensures that analytical aggregations such as GROUP BY category or SUM by city produce consistent results regardless of the query path taken.

1.2 Entity-Relationship Diagram (ERD) and Schema Architecture

The Entity-Relationship Diagram (ERD) constitutes the master blueprint of the database schema, providing a graphical representation of all entities (tables), their attributes (columns), and the relationships (foreign key constraints) that bind them. The ERD for this project was constructed using Microsoft SQL Server Management Studio's (SSMS) built-in diagram tool and encompasses fourteen core entities interconnected through twenty-two foreign key relationships. The diagram provides both a high-level architectural overview and a precise technical specification of the referential integrity constraints enforced at the database engine level.

The ERD illustrates the hierarchical dependency structure of the schema, with the Customers table serving as the root anchor entity from which multiple downstream entities derive their existential context. The following subsections document each entity and its role within the broader relational model.

1.2.1 Core Entity Descriptions

The Customers table is the central entity in the schema and stores the demographic and authentication profile of each registered platform user. Its primary key, CustomerID, is an auto-incrementing integer identity column that serves as the foreign key anchor in six dependent tables: CustomerAddresses, Carts, Orders, Returns, SupportTickets, and implicitly through the order chain — Payments, Invoices, Shipments, and Refunds. Key attributes include FirstName and LastName (nvarchar(50)), Email (nvarchar(100) with a UNIQUE constraint preventing duplicate registrations), Phone (nvarchar(20), nullable to accommodate customers who register without a phone number), PasswordHash (nvarchar(255) storing a cryptographically hashed credential), RegistrationDate (datetime2(7) defaulting to GETDATE() to capture precise registration timestamps), and IsActive (bit, defaulting to 1) which serves as the soft-deletion flag for customer account deactivation.

The Products table governs the master product catalog and maintains a direct dependency on the Categories table through a non-nullable CategoryID foreign key. Each product is identified by a system-generated ProductID and a business-defined SKU (nvarchar(60) with a UNIQUE constraint) that serves as the human-readable product identifier for inventory and order management workflows. Financial attributes include Price (the retail selling price) and CostPrice (the procurement cost), both stored as decimal(10,2) to ensure two-decimal-place precision for monetary values — a critical requirement for accurate gross margin calculations. The StockQuantity integer field tracks real-time inventory levels, while LowStockThreshold (defaulting to 10) enables automated low-stock alerting. The IsActive bit flag supports product delisting without physical record deletion.

The Orders table is the transactional hub of the schema, recording each completed purchase event. It references CustomerID (linking to the purchasing customer), CartID (linking to the originating shopping cart), and ShippingAddressID (linking to the selected delivery address from CustomerAddresses). The OrderDate field defaults to GETDATE(), the Status field (nvarchar(20)) defaults to 'Pending' and transitions through states including 'Processing', 'Shipped', 'Delivered', and 'Cancelled', and TotalAmount records the final computed order value. The Notes field (nvarchar(max)) provides an unstructured annotation field for customer or operator remarks.

The OrderItems table implements the many-to-many resolution between Orders and Products, with each row representing a single product line within a multi-item order. It captures the OrderID, ProductID, Quantity, and UnitPrice at the time of purchase (preserving the historical price irrespective of subsequent price changes), and computes a PERSISTED computed column $\text{LineTotal} = \text{Quantity} * \text{UnitPrice}$ that is stored physically on disk for query performance optimization.

The Payments table maintains a 1:1 relationship with Orders, recording the payment method (nvarchar(20)), transaction amount (decimal(10,2)), payment status (defaulting to 'Pending' and transitioning through 'Paid', 'Failed', 'Refunded'), the precise payment timestamp (PaidAt, datetime2(7), nullable to accommodate deferred payment scenarios), and a TransactionRef field (nvarchar(100)) storing the external payment gateway transaction identifier.

The Invoices table captures the formal billing documents associated with each order, maintaining a 1:1 relationship with Orders through a UNIQUE constraint on the OrderID foreign key. It generates a sequential InvoiceNumber (nvarchar(50) with UNIQUE constraint), records the issuance timestamp (IssuedAt, defaulting to GETDATE()), specifies an optional DueDate for deferred payment invoices, and captures the TotalAmount (decimal(10,2)) at the time of invoice generation.

The Shipments table records the logistics lifecycle of each dispatched order, linking to both the Orders table (OrderID foreign key) and the Couriers table (CourierID foreign key). It captures a unique TrackingNumber, shipment Status (defaulting to 'Pending'), ShippedDate, EstimatedExpectedDate (date type for day-level precision), and DeliveredDate (datetime2(7) for precise delivery timestamping).

The Couriers table maintains the master registry of logistics partners, storing CourierName (nvarchar(150)) and ContactInfo (nvarchar(150)) as basic operational reference data. The Returns table records post-delivery product return requests, linking to Orders (OrderID), Customers (CustomerID), and Products (ProductID) to enable multi-dimensional return analysis. It captures QuantityReturned, ReturnReason (nvarchar(255)), ReturnDate, and a Status field (defaulting to 'Requested') tracking the return lifecycle. The Refunds table extends the Returns chain with financial resolution data: RefundAmount (decimal(10,2)), RefundStatus (defaulting to 'Pending'), and RefundDate.

The SupportTickets table captures customer service interactions, linking to Customers through CustomerID and storing Subject (nvarchar(200)), Description (nvarchar(max)), Priority

(nvarchar(10) defaulting to 'Medium'), Status (nvarchar(20) defaulting to 'Open'), CreatedAt (defaulting to GETDATE()), and ClosedAt. The Categories table provides the hierarchical product taxonomy, with a self-referencing ParentCategoryID foreign key enabling nested category structures. The CustomerAddresses and Carts tables complete the schema, providing the address registry and shopping session persistence mechanisms respectively.

1.2.2 Foreign Key Relationship Inventory

The following table provides a comprehensive inventory of all foreign key relationships defined in the schema, documenting the parent table, child table, and the linking key column for each constraint. These relationships collectively enforce referential integrity across the entire schema, preventing orphaned records and ensuring that every transaction can be traced back to its originating customer, product, and order context.

Constraint Name	Parent Table	Child Table	Foreign Key Column
FK_Product_Category	Categories	Products	CategoryID
FK_Category_Parent	Categories	Categories	ParentCategoryID
FK_Address_Customer	Customers	CustomerAddresses	CustomerID
FK_Cart_Customer	Customers	Carts	CustomerID
FK_CartItem_Cart	Carts	CartItems	CartID
FK_CartItem_Product	Products	CartItems	ProductID
FK_Order_Customer	Customers	Orders	CustomerID
FK_Order_Cart	Carts	Orders	CartID
FK_Order_Address	CustomerAddresses	Orders	ShippingAddressID
FK_OrderItem_Order	Orders	OrderItems	OrderID
FK_OrderItem_Product	Products	OrderItems	ProductID
FK_Payment_Order	Orders	Payments	OrderID
FK_Invoice_Order	Orders	Invoices	OrderID
FK_Return_Order	Orders	Returns	OrderID
FK_Return_Customer	Customers	Returns	CustomerID
FK_Return_Product	Products	Returns	ProductID
FK_Refund_Return	Returns	Refunds	ReturnID
FK_Shipment_Order	Orders	Shipments	OrderID
FK_Shipment_Courier	Couriers	Shipments	CourierID
FK_Ticket_Customer	Customers	SupportTickets	CustomerID

1.3 Data Definition Language (DDL) Implementation

The physical instantiation of the database schema was accomplished through a meticulously structured set of Data Definition Language (DDL) statements executed in sequential order within Microsoft SQL Server Management Studio (SSMS). The DDL script was organized into three logical execution blocks: first, the creation of all base tables without foreign key constraints (to avoid circular dependency errors during initial table creation); second, the application of all ALTER TABLE ADD CONSTRAINT statements to establish the foreign key relationships; and third, the creation of database views that encapsulate commonly used multi-table join patterns for query performance optimization.

1.3.1 Table Creation — Base Entities

The table creation sequence followed a dependency-ordered execution plan, beginning with the leaf entities that carry no foreign key dependencies and progressing toward the hub entities that reference multiple parent tables. The Categories and Couriers tables were created first as they carry no inbound foreign key dependencies. The Customers table followed, establishing the primary customer identity anchor. Products was created after Categories to enable the immediate CategoryID foreign key specification.

Several DDL design decisions warrant specific technical commentary. The universal adoption of IDENTITY(1,1) for all primary key columns ensures auto-incrementing integer key generation starting from 1 with an increment of 1, eliminating the need for application-layer key management and guaranteeing uniqueness even under concurrent insertion scenarios. The datetime2(7) data type was selected over the legacy datetime type for all timestamp columns due to its superior precision (100-nanosecond resolution versus 3.33-millisecond resolution for datetime), its broader date range (January 1, 0001 through December 31, 9999), and its compatibility with the ISO 8601 standard.

Monetary values across all financial tables (Orders.TotalAmount, Payments.Amount, Invoices.TotalAmount, OrderItems.UnitPrice, Products.Price, Products.CostPrice, Refunds.RefundAmount) were uniformly typed as decimal(10,2), providing ten total digits with two decimal places of precision. This choice over the float or money data types eliminates floating-point rounding errors — a critical requirement for financial data where even sub-cent discrepancies can aggregate to significant reconciliation issues at scale.

The UNIQUE constraint applied to Customers.Email prevents the creation of duplicate customer accounts sharing the same email address, a common integrity requirement in authentication

systems. Similarly, the UNIQUE constraint on Products.SKU ensures that each product's stock-keeping unit identifier remains globally distinct across the product catalog, enabling reliable inventory synchronization with external warehouse management systems.

1.3.2 Computed Columns and Constraints

A technically significant DDL feature employed in this schema is the PERSISTED computed column LineTotal in the OrderItems table, defined as `AS ([Quantity]*[UnitPrice]) PERSISTED`. In SQL Server, a persisted computed column is physically stored on disk rather than being calculated at query runtime, meaning that SELECT queries on LineTotal benefit from index support and avoid redundant multiplication operations during analytical aggregations. This design choice reflects performance optimization awareness — in a dataset with thousands of order items, the elimination of runtime computation for the most frequently aggregated financial field provides meaningful query latency improvements.

The DEFAULT constraint applied to `Orders.OrderDate = GETDATE()` and `Customers.RegistrationDate = GETDATE()` ensures that timestamp fields are automatically populated with the server-side current datetime at the moment of record insertion, eliminating the risk of NULL timestamps arising from application-layer oversight and providing a reliable audit trail for all transactional records.

1.3.3 Foreign Key Relationship Establishment

Following the successful creation of all base tables, a second DDL execution block applied the complete set of foreign key constraints through ALTER TABLE ADD CONSTRAINT statements. The deliberate separation of table creation from constraint application was necessitated by circular dependency scenarios — for instance, the Categories table self-references through ParentCategoryID, and the Orders table references both Customers and Carts, which themselves reference Customers. By creating all tables before applying any constraints, the execution plan avoids the runtime errors that would arise if SQL Server attempted to validate a foreign key reference to a table that had not yet been created.

Each foreign key constraint was assigned a descriptive, standardized name following the convention `FK_ChildTable_ParentTable` (e.g., `FK_Product_Category`, `FK_Order_Customer`), facilitating both schema documentation and the identification of constraint violations in error messages. No ON DELETE CASCADE or ON UPDATE CASCADE actions were specified for any foreign key relationship, reflecting the deliberate architectural decision that all referential

integrity violations should surface as explicit errors rather than being silently resolved through cascading operations — a conservative but auditably sound approach for financial transactional data.

1.3.4 Database Views

To encapsulate complex multi-table join logic and provide simplified query interfaces for the analytical and reporting layers, several SQL views were created as part of the DDL implementation. Views serve as virtual tables that present pre-defined query results as named, reusable objects, reducing query complexity for end-users and providing a stable API surface that abstracts underlying schema changes.

The `vw_CustomerMaster` view consolidates customer demographic data with order statistics, joining the `Customers` table with `CustomerAddresses` (on `IsDefault = 1` to retrieve the primary address), `Orders` (excluding Cancelled status), and `Payments` (filtering for Paid status) to produce a denormalized record per customer containing `FullName`, `Email`, `Phone`, `City`, `RegistrationDate`, `TotalOrders` count, and `TotalSpend` aggregation. This view directly powers the 'Top Premium Customers' SQL query and the customer-level Power BI visuals.

The `vw_ProductCatalog` view joins `Products` with `Categories` to expose `CategoryName` alongside all product attributes, and applies a conditional CASE statement to derive a `StockStatus` field ('Out of Stock' when `StockQuantity <= 0`, 'Low Stock' when `StockQuantity <= LowStockThreshold`, 'In Stock' otherwise). This view enables single-query inventory status monitoring without requiring the calling query to embed the stock threshold logic.

The `vw_OrderDetails` view implements a comprehensive five-table join across `OrderItems`, `Orders`, `Customers`, `Products`, `Payments`, and `Shipments`, producing a fully denormalized order line record that includes customer name, order date, product name, quantity, unit price, line total, payment status, and shipment status. This master view serves as the primary data source for the Power BI dashboards and the Comprehensive Order Report SQL query.

The `vw_ReturnRefundSummary` view aggregates the `Returns` and `Refunds` tables to produce a per-customer, per-product return analysis, while the `vw_SupportSLA` view computes SLA breach flags for each support ticket based on priority-differentiated resolution time thresholds. The `vw_DailySalesSummary` view groups orders by date to provide a rolling daily KPI summary of total orders, revenue, and items sold.

1.4 Data Ingestion Strategy and DML Operations

With the complete schema established, the second operational phase involved populating the fourteen tables with representative data at a scale sufficient to enable statistically meaningful analysis. The data ingestion strategy employed two complementary approaches: BULK INSERT operations for high-volume table population from pre-formatted CSV files, and standard T-SQL INSERT statements for transactional CRUD (Create, Read, Update, Delete) capability demonstration.

1.4.1 BULK INSERT Implementation

The BULK INSERT command in SQL Server provides a high-performance, server-side batch loading mechanism that reads directly from formatted flat files (CSV, TSV, or custom-delimited formats) and inserts records into target tables at significantly higher throughput than row-by-row INSERT statements. For this project, the BULK INSERT approach was essential given the dataset size: loading 22,049 customer behavior records through individual INSERT statements would have been prohibitively time-consuming and would have generated excessive transaction log activity.

Each BULK INSERT command was configured with the following parameters: `FORMAT = 'CSV'` to specify comma-delimited input; `FIRSTROW = 2` to skip the header row present in all source CSV files; `FIELDTERMINATOR = ','` to designate the comma as the column separator; `ROWTERMINATOR = '\n'` to specify Unix-style newline as the row delimiter; `TABLOCK` to acquire a table-level lock during the bulk operation, improving throughput by reducing lock management overhead; and `KEEPIDENTITY` to preserve the identity values specified in the source CSV rather than generating new auto-increment values — critical for maintaining referential integrity between related tables loaded from separate CSV files.

The BULK INSERT execution sequence was carefully ordered to respect the foreign key dependency hierarchy established by the schema constraints. Leaf-level tables with no inbound foreign key dependencies (Categories, Couriers, Customers, Products) were loaded first. Intermediate tables (CustomerAddresses, Carts) were loaded second. Transactional tables (Orders, Payments, Invoices, OrderItems, CartItems, Shipments, Returns, Refunds, SupportTickets) were loaded in the final phase, after all their referenced parent records had been successfully inserted.

1.4.2 Post-Ingestion Validation

Following the completion of all BULK INSERT operations, a validation script was executed to verify the integrity of the loaded data. This script performed row count checks on each table to

confirm that the expected record counts matched the source CSV row counts, checked for NULL values in mandatory non-nullable columns, and executed a series of referential integrity probe queries to confirm that no orphaned records existed in child tables. The validation confirmed zero data loss across all fourteen tables, zero NULL violations in mandatory fields, and zero orphaned records in any child table.

Additionally, the PRINT statements appended to each BULK INSERT block ('Products loaded', 'Payments loaded', etc.) provided runtime confirmation messages in the SSMS Messages pane, enabling the operator to monitor progress and identify any load failures immediately. The final confirmation message 'Query executed successfully' displayed in the SSMS status bar, accompanied by the 10-second execution timestamp recorded for the full script, provided definitive evidence of successful end-to-end data ingestion.

1.4.3 Standard DML Operations

In addition to the bulk loading operations, the project demonstrated standard DML capabilities through individual INSERT, UPDATE, SELECT, and logical DELETE (status update) operations on representative data records. These operations were implemented to validate the transactional integrity of the schema and to confirm that the foreign key constraints operated correctly under live data manipulation conditions.

A representative UPDATE operation modified the Status of a specific order from 'Pending' to 'Cancelled', triggering the soft-deletion logic described in the business rules section. A SELECT operation with a WHERE clause filtered the Orders table to return only records with Status = 'Cancelled', confirming that the update had been correctly applied. An INSERT operation added a new customer record with a complete address entry, validating that the cascading address-to-customer foreign key relationship permitted the dependent insert after the parent customer record had been committed.

1.5 Business Analytics SQL Queries

With the database fully populated, a suite of ten analytical SQL queries was developed to extract direct business intelligence metrics. These queries were designed to answer specific operational questions posed by the project's business stakeholder personas — the marketing team, inventory managers, finance department, and C-suite executives. The queries collectively demonstrate the full spectrum of SQL analytical capabilities: scalar aggregation, grouped aggregation, multi-table joins, subqueries, window functions, Common Table Expressions (CTEs), and conditional logic.

1.5.1 Query 1 — Active Customer Master View

The first query materialized the `vw_CustomerMaster` view and retrieved the top 20 customers ordered by `TotalSpend` in descending order. The underlying view logic employed a `LEFT JOIN` from `Customers` to `CustomerAddresses` (on `CustomerID` and `IsDefault = 1` to retrieve the primary address only), an `OUTER JOIN` to `Orders` (excluding `Cancelled` status), and an `OUTER JOIN` to `Payments` (filtering for `Paid` status), with `GROUP BY` on the customer identifier. The `SELECT TOP 20 ... ORDER BY TotalSpend DESC` clause surfaced the highest lifetime spenders. This query feeds directly into the marketing team's VIP customer identification workflow and provides the input data for the loyalty program targeting system.

1.5.2 Query 2 — Total Sales by Product Category

The second query used `SUM()` and `GROUP BY` in conjunction with the `vw_ProductCatalog` view and the `OrderItems` table to calculate total revenue by product category. The query joined `OrderItems` to `Products` to `Categories`, grouped by `CategoryName`, and sorted by `TotalRevenue` in descending order. The result confirmed that `Electronics` leads category revenue with approximately \$3.7 million in total sales across the dataset period, followed by `Home & Garden` at \$1.6 million and `Sports` at \$1.1 million. This finding corroborated the Python EDA results and provided the authoritative SQL-sourced revenue breakdown for the Power BI dashboards.

1.5.3 Query 3 — Daily Sales KPI Summary

The third query computed a rolling daily sales summary by grouping orders on a date-cast `OrderDate` field and aggregating `COUNT(DISTINCT OrderID)` as `OrdersCount`, `ISNULL(SUM(Payment.Amount), 0)` as `TotalRevenue`, and the Average Order Value as `TotalRevenue / NULLIF(OrdersCount, 0)`. A `WHERE` clause restricted results to orders placed within the last 30 days relative to `GETDATE()`, providing a rolling operational view relevant for daily business monitoring. The results exposed specific high-revenue days coinciding with promotional campaign dates, validating the value of daily-granularity monitoring.

1.5.4 Query 4 — Order Distribution by City

The fourth query extracted geographical order distribution by joining Orders to Customers (on CustomerID) and grouping by the customer City attribute. The COUNT(OrderID) function quantified total orders per city, and the SUM(TotalAmount) function computed total revenue per city. The results revealed that Istanbul, the most populous city in the dataset, accounted for 1,672 orders — the largest single-city order volume. This geographical distribution insight directly informs warehouse location optimization and last-mile delivery routing decisions.

1.5.5 Queries 5 through 10 — Extended Analytics Suite

Query 5 identified top-selling products by quantity through a join between OrderItems and Products, aggregating SUM(Quantity) and ordering descending to surface the highest-velocity inventory items. Query 6 leveraged a window function (DENSE_RANK() OVER PARTITION BY CategoryName ORDER BY SUM(Quantity) DESC) to rank products within each category by units sold, providing a within-category performance leaderboard. Query 7 performed Returns and Refunds analytics, computing COUNT(ReturnID) grouped by ReturnReason to identify the most common return motivations — the top five being Better Price Elsewhere (63 returns), Not as Described (59), Wrong Item Sent (59), Defective Product (58), and Missing Parts (57).

Query 8 analyzed the Cart-to-Order Conversion rate, revealing that of 3,500 carts created, 3,000 converted to orders — a conversion rate of 85.71%. This metric is a critical e-commerce KPI that measures the effectiveness of the checkout funnel. Query 9 computed the Delivery Performance and Delay Report per courier by joining Shipments to Couriers, counting total shipments, on-time deliveries, late deliveries, and average delivery days. The results indicated that all four couriers (Aramex, DHL, FedEx, UPS) achieved 100% on-time delivery rates within the dataset, with an average delivery duration of approximately one day — suggesting that the simulated dataset reflects an optimistic logistics scenario suitable for establishing a performance baseline.

Query 10 generated a Comprehensive Revenue by Payment Method report, categorizing transactions as TotalPaid (Paid status), FailedOrCancelledCount (Failed or Cancelled status), and TotalOutstanding (Delivered but unpaid). The results indicated that Card payments (Credit Card combined) represented the majority payment mechanism, providing the payment gateway team with the volume data necessary for merchant fee negotiation.

1.6 Chapter Summary and Transition to Exploratory Data Analysis

This chapter has presented a comprehensive technical documentation of the relational database management system designed, implemented, and populated as the data infrastructure foundation

of the E-Commerce Store Management System project. The 14-table, 22-constraint schema was constructed in accordance with Third Normal Form normalization principles, encoded fourteen explicit business rules as structural and constraint-level enforcement mechanisms, and was populated through a rigorous BULK INSERT pipeline that loaded 22,049+ transactional records without data loss or referential integrity violation.

The SQL analytical query suite demonstrated the full spectrum of SQL Server's analytical capabilities — from basic SELECT-WHERE filtering through multi-table JOIN aggregations and window function rankings to CTE-based complex reporting. The business insights extracted through these queries — including Electronics category revenue dominance, an 85.71% cart conversion rate, the geographical concentration of orders in Istanbul, and the identification of top-spending VIP customers — provide the initial analytical baseline against which the deeper statistical and machine learning analyses of subsequent phases will be contextualized.

The structured, normalized relational schema constructed in this phase serves as the authoritative data source for all downstream analytical processes: it feeds the Python EDA Jupyter Notebook through CSV exports, the Orange Data Mining pipeline through structured dataset imports, and the Power BI dashboards through a live SQL Server connection. The architectural decisions made in this phase — particularly the separation of order history from customer profiles, the soft-deletion approach to cancelled orders, and the PERSISTED computed column for order line totals — reflect professional-grade database engineering judgment that directly enhances both the performance and the analytical fidelity of all subsequent project phases.

Chapter 1 Summary Statistics:

Database Tables Created:	14 normalized tables
Foreign Key Constraints:	22 referential integrity constraints
Database Views Created:	6 analytical views
Records Ingested (Total):	22,049+ across all tables
Analytical Queries Written:	10 business-intelligence queries
Cart-to-Order Conversion:	85.71%
Top Revenue Category:	Electronics (\$3.7M total)
Top City by Orders:	Istanbul (1,672 orders)

